

Sequential Functional Optimization, Loss Margin Maximization, and Perturbative Corrections

A Mathematical, Information-Theoretic, and Physical Foundation of Boosting

Contents

1	The Sequential Boosting Paradigm and Functional Optimization	2
1.1	PAC Learning and the Weak-Learnability Equivalence	2
1.2	Forward Stagewise Additive Modeling	2
2	Adaptive Boosting (AdaBoost): Exponential Loss Minimization	3
2.1	Derivation of AdaBoost from Exponential Loss	3
2.2	Proof of Training Error Bound	4
3	Gradient Boosting: Functional Gradient Descent	5
3.1	Gradient Descent in Hilbert Function Spaces	5
3.2	Projection onto the Restricted Hypothesis Class	5
3.3	Line Search and Region-Based Multi-Step Updates	6
3.4	Derivation of Pseudo-Residuals for Common Loss Functions	6
4	Information-Theoretic View and Physical Analogies	7
4.1	Information-Theoretic View: Greedy KL-Divergence Minimization	7
4.2	Physical Analogy: Rayleigh-Schrödinger Perturbation Theory	7
5	Regularization, Shrinkage, and Generalization Dynamics	8
5.1	Shrinkage (Learning Rate Control)	8
5.2	Stochastic Gradient Boosting (Subsampling)	8
5.3	Margin Maximization Theory	8
6	Production-Grade Vectorized NumPy Implementation	9

1 The Sequential Boosting Paradigm and Functional Optimization

Unlike variance-reduction techniques that average parallel predictors trained independently on re-sampled datasets, **Boosting** is a sequential, additive optimization paradigm. It transforms a sequence of weak base hypotheses—models performing only slightly better than random guessing—into a strong predictor with arbitrarily small generalization error.

Introduced conceptually by Kearns and Valiant (1988) and realized practically through Schapire (1990) and Freund & Schapire (1997), boosting reframes supervised learning as **gradient descent** in function space.

Definition 1.1 (Additive Functional Model). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be a training set of N i.i.d. observations sampled from $p_{\text{data}}(\mathbf{x}, y)$. An **Additive Functional Model** $f_M(\mathbf{x})$ represents a linear combination of M base functions $h(\mathbf{x}; \boldsymbol{\theta}_m)$ drawn from a restricted hypothesis class $\mathcal{H}$:

$$f_M(\mathbf{x}) = \sum_{m=1}^M \beta_m h(\mathbf{x}; \boldsymbol{\theta}_m) \quad (1)$$

where $\beta_m \in \mathbb{R}$ represents the expansion coefficient (stage weight) and $\boldsymbol{\theta}_m$ dictates the structural parameters of the m -th base component $h(\cdot; \boldsymbol{\theta}_m)$.

1.1 PAC Learning and the Weak-Learnability Equivalence

The theoretical underpinnings of boosting rest on **Probably Approximately Correct (PAC) Learning Theory**.

Definition 1.2 (Weak vs. Strong PAC Learnability). A concept class $\mathcal{C}$ is **Strongly PAC Learnable** if there exists an algorithm $\mathcal{A}$ such that for any distribution P over $\mathcal{X}$, and any $\epsilon > 0, \delta > 0$, $\mathcal{A}$ outputs a hypothesis h satisfying $P(L(h) > \epsilon) < \delta$ in polynomial time.

A concept class $\mathcal{C}$ is **Weakly PAC Learnable** if the same conditions hold strictly for a fixed, loose error bound $\epsilon < \frac{1}{2} - \gamma$, where $\gamma > 0$ represents a marginal edge over random guessing.

Theorem 1.1 (Schapire’s Equivalence Theorem, 1990). A concept class $\mathcal{C}$ is weakly PAC learnable if and only if it is strongly PAC learnable.

Boosting constructs a constructive proof of this equivalence: by sequentially reweighting instance distributions or fitting residual gradients, a sequence of weak learners with $\epsilon_m = \frac{1}{2} - \gamma_m$ converges to a strong model with arbitrarily low empirical risk.

1.2 Forward Stagewise Additive Modeling

Directly optimizing the joint loss over all M stage weights and parameter vectors simultaneously is computationally intractable:

$$\min_{\{\beta_m, \boldsymbol{\theta}_m\}_{m=1}^M} \sum_{i=1}^N \mathcal{L} \left(y^{(i)}, \sum_{m=1}^M \beta_m h(\mathbf{x}^{(i)}; \boldsymbol{\theta}_m) \right) \quad (2)$$

Boosting approximates this global objective using **Forward Stagewise Additive Modeling**. At iteration m , the model fixes all previous components $f_{m-1}(\mathbf{x})$ and greedily optimizes only the current stage parameters $(\beta_m, \boldsymbol{\theta}_m)$:

$$(\beta_m, \boldsymbol{\theta}_m) = \arg \min_{\beta, \boldsymbol{\theta}} \sum_{i=1}^N \mathcal{L} \left(y^{(i)}, f_{m-1}(\mathbf{x}^{(i)}) + \beta h(\mathbf{x}^{(i)}; \boldsymbol{\theta}) \right) \quad (3)$$

The current hypothesis is then updated additively:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m h(\mathbf{x}; \boldsymbol{\theta}_m) \quad (4)$$

2 Adaptive Boosting (AdaBoost): Exponential Loss Minimization

****AdaBoost (Adaptive Boosting)****, developed by Freund and Schapire (1997), solves binary classification tasks where $y^{(i)} \in \{-1, +1\}$ by sequentially updating an instance weight distribution $w_i^{(m)}$ over the training set.

2.1 Derivation of AdaBoost from Exponential Loss

We prove that AdaBoost is an exact implementation of forward stagewise additive modeling under the ****Exponential Loss Function****:

$$\mathcal{L}(y, f(\mathbf{x})) = \exp(-yf(\mathbf{x})) \quad (5)$$

Theorem 2.1 (AdaBoost Stage-Wise Equivalence). Forward stagewise additive optimization of exponential loss yields the exact parameter updates and instance weighting scheme of the AdaBoost algorithm.

Proof. At iteration m , we seek to minimize the objective function $\mathcal{J}_m(\beta, h)$:

$$\mathcal{J}_m(\beta, h) = \sum_{i=1}^N \exp\left(-y^{(i)} \left(f_{m-1}(\mathbf{x}^{(i)}) + \beta h(\mathbf{x}^{(i)})\right)\right) \quad (6)$$

Factoring out the term corresponding to the accumulated predictor $f_{m-1}(\mathbf{x}^{(i)})$:

$$\mathcal{J}_m(\beta, h) = \sum_{i=1}^N \underbrace{\exp\left(-y^{(i)} f_{m-1}(\mathbf{x}^{(i)})\right)}_{w_i^{(m)}} \exp\left(-y^{(i)} \beta h(\mathbf{x}^{(i)})\right) \quad (7)$$

where $w_i^{(m)}$ is an unnormalized sample weight assigned to instance i at stage m .

Assuming $h(\mathbf{x}) \in \{-1, +1\}$, we split the summation into correctly classified samples ($y^{(i)} = h(\mathbf{x}^{(i)})$) and misclassified samples ($y^{(i)} \neq h(\mathbf{x}^{(i)})$):

$$\begin{aligned} \mathcal{J}_m(\beta, h) &= \sum_{y^{(i)}=h(\mathbf{x}^{(i)})} w_i^{(m)} e^{-\beta} + \sum_{y^{(i)} \neq h(\mathbf{x}^{(i)})} w_i^{(m)} e^{\beta} \\ &= e^{-\beta} \sum_{i=1}^N w_i^{(m)} + (e^{\beta} - e^{-\beta}) \sum_{y^{(i)} \neq h(\mathbf{x}^{(i)})} w_i^{(m)} \end{aligned} \quad (8)$$

Let $W^{(m)} = \sum_{i=1}^N w_i^{(m)}$ be the total sample weight, and define the weighted error rate ϵ_m :

$$\epsilon_m = \frac{\sum_{y^{(i)} \neq h(\mathbf{x}^{(i)})} w_i^{(m)}}{\sum_{i=1}^N w_i^{(m)}} = \sum_{i=1}^N \tilde{w}_i^{(m)} \mathbb{I}(y^{(i)} \neq h(\mathbf{x}^{(i)})) \quad (9)$$

where $\tilde{w}_i^{(m)} = w_i^{(m)} / W^{(m)}$ represents the normalized weight distribution.

Substituting $\epsilon_m W^{(m)}$ into the objective:

$$\mathcal{J}_m(\beta, h) = W^{(m)} \left[(1 - \epsilon_m) e^{-\beta} + \epsilon_m e^{\beta} \right] \quad (10)$$

To find optimal base learner $h_m(\mathbf{x})$, notice that for any fixed $\beta > 0$, minimizing $\mathcal{J}_m(\beta, h)$ is strictly equivalent to minimizing weighted error rate ϵ_m :

$$h_m(\mathbf{x}) = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N \tilde{w}_i^{(m)} \mathbb{I}(y^{(i)} \neq h(\mathbf{x}^{(i)})) \quad (11)$$

Having determined $h_m(\mathbf{x})$, we optimize stage coefficient β_m by taking the partial derivative with respect to β and setting it to zero:

$$\frac{\partial \mathcal{J}_m}{\partial \beta} = W^{(m)} \left[-(1 - \epsilon_m)e^{-\beta} + \epsilon_m e^{\beta} \right] = 0 \quad (12)$$

$$(1 - \epsilon_m)e^{-\beta_m} = \epsilon_m e^{\beta_m} \implies e^{2\beta_m} = \frac{1 - \epsilon_m}{\epsilon_m} \quad (13)$$

Taking the natural logarithm yields the optimal stage weight $\alpha_m = 2\beta_m$:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right) \quad (14)$$

Finally, updating the accumulated prediction function $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m h_m(\mathbf{x})$ updates sample weights for the next iteration:

$$w_i^{(m+1)} = w_i^{(m)} \exp \left(-y^{(i)} \beta_m h_m(\mathbf{x}^{(i)}) \right) = w_i^{(m)} \exp \left(-\frac{1}{2} \alpha_m y^{(i)} h_m(\mathbf{x}^{(i)}) \right) \quad (15)$$

This completes the derivation of the exact AdaBoost update rules. $\square$

2.2 Proof of Training Error Bound

A notable property of AdaBoost is its ability to drive empirical training error to zero exponentially fast as M grows.

Theorem 2.2 (AdaBoost Training Error Bound). The empirical classification error $R_{\text{emp}}(f_M) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y^{(i)} \neq \text{sign}(f_M(\mathbf{x}^{(i)})))$ of the AdaBoost model satisfies the upper bound:

$$R_{\text{emp}}(f_M) \leq \prod_{m=1}^M 2\sqrt{\epsilon_m(1 - \epsilon_m)} = \exp \left(-2 \sum_{m=1}^M \gamma_m^2 \right) \quad (16)$$

where $\gamma_m = \frac{1}{2} - \epsilon_m$ represents the edge over random guessing.

Proof. Because $0 \leq \mathbb{I}(y \neq \text{sign}(f(\mathbf{x}))) \leq \exp(-yf(\mathbf{x}))$ for all $y \in \{-1, +1\}$:

$$R_{\text{emp}}(f_M) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y^{(i)} f_M(\mathbf{x}^{(i)}) \leq 0) \leq \frac{1}{N} \sum_{i=1}^N \exp \left(-y^{(i)} f_M(\mathbf{x}^{(i)}) \right) \quad (17)$$

By definition of the weight update rule $w_i^{(m+1)} = w_i^{(m)} e^{-y^{(i)} \beta_m h_m(\mathbf{x}^{(i)})}$, telescoping across all M stages yields:

$$\frac{1}{N} \sum_{i=1}^N \exp \left(-y^{(i)} f_M(\mathbf{x}^{(i)}) \right) = \frac{1}{N} \sum_{i=1}^N w_i^{(M+1)} = \frac{W^{(M+1)}}{N} \quad (18)$$

Evaluating the normalization factor ratio $Z_m = \frac{W^{(m+1)}}{W^{(m)}}$:

$$\begin{aligned} Z_m &= \frac{W^{(m+1)}}{W^{(m)}} = \sum_{i=1}^N \tilde{w}_i^{(m)} \exp \left(-\beta_m y^{(i)} h_m(\mathbf{x}^{(i)}) \right) \\ &= (1 - \epsilon_m)e^{-\beta_m} + \epsilon_m e^{\beta_m} \end{aligned} \quad (19)$$

Substituting $\beta_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$:

$$Z_m = (1 - \epsilon_m) \sqrt{\frac{\epsilon_m}{1 - \epsilon_m}} + \epsilon_m \sqrt{\frac{1 - \epsilon_m}{\epsilon_m}} = 2\sqrt{\epsilon_m(1 - \epsilon_m)} \quad (20)$$

Accumulating the product of normalization factors $W^{(M+1)} = N \prod_{m=1}^M Z_m$:

$$R_{\text{emp}}(f_M) \leq \prod_{m=1}^M Z_m = \prod_{m=1}^M 2\sqrt{\epsilon_m(1-\epsilon_m)} \quad (21)$$

Substituting $\epsilon_m = \frac{1}{2} - \gamma_m$:

$$2\sqrt{\epsilon_m(1-\epsilon_m)} = 2\sqrt{\left(\frac{1}{2} - \gamma_m\right)\left(\frac{1}{2} + \gamma_m\right)} = \sqrt{1 - 4\gamma_m^2} \quad (22)$$

Using the inequality $1 - x \leq e^{-x}$ for $x = 4\gamma_m^2$:

$$\sqrt{1 - 4\gamma_m^2} \leq \left(e^{-4\gamma_m^2}\right)^{1/2} = e^{-2\gamma_m^2} \implies R_{\text{emp}}(f_M) \leq \exp\left(-2 \sum_{m=1}^M \gamma_m^2\right) \quad (23)$$

This completes the proof. $\square$

3 Gradient Boosting: Functional Gradient Descent

While AdaBoost minimizes exponential loss over binary labels, ****Gradient Boosting**** (Friedman, 2001) generalizes boosting to arbitrary differentiable loss functions $\mathcal{L}(y, f(\mathbf{x}))$ over continuous regression, multi-class classification, or ranking objectives.

3.1 Gradient Descent in Hilbert Function Spaces

Consider optimizing an empirical loss functional $\mathcal{C}(f)$ over an infinite-dimensional function space $\mathcal{H}$:

$$\mathcal{C}(f) = \sum_{i=1}^N \mathcal{L}\left(y^{(i)}, f(\mathbf{x}^{(i)})\right) \quad (24)$$

In standard parameter-space gradient descent, parameters $\boldsymbol{\theta}$ update along the negative loss gradient. In ****Functional Gradient Descent****, we treat the evaluation vector $\mathbf{f} = [f(\mathbf{x}^{(1)}), \dots, f(\mathbf{x}^{(N)})]^T \in \mathbb{R}^N$ as the parameter space itself.

The unconstrained negative gradient direction $\mathbf{g}_m \in \mathbb{R}^N$ at stage m points along the steepest reduction of loss evaluated at the current hypothesis $f_{m-1}(\mathbf{x})$:

$$g_{im} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, f(\mathbf{x}^{(i)}))}{\partial f(\mathbf{x}^{(i)})} \right]_{f(\mathbf{x})=f_{m-1}(\mathbf{x})} \quad (25)$$

Definition 3.1 (Pseudo-Residuals). The scalar components g_{im} of the functional negative gradient are termed ****Pseudo-Residuals****. They define the direction and magnitude of response change needed at each training point to minimize the loss.

3.2 Projection onto the Restricted Hypothesis Class

The raw negative gradient vector $\mathbf{g}_m \in \mathbb{R}^N$ is defined only at discrete training points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$, failing to generalize to unobserved query points $\mathbf{x}$.

To generalize, Gradient Boosting fits a base learner $h_m(\mathbf{x}; \boldsymbol{\theta}_m) \in \mathcal{H}$ to match the pseudo-residuals g_{im} via least-squares structural regression:

$$\boldsymbol{\theta}_m = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \left(g_{im} - h(\mathbf{x}^{(i)}; \boldsymbol{\theta}) \right)^2 \quad (26)$$

Fitting $h_m(\mathbf{x})$ acts as a constrained inner-product projection of the unconstrained gradient $\mathbf{g}_m$ onto the span of restricted hypothesis class $\mathcal{H}$.

3.3 Line Search and Region-Based Multi-Step Updates

Having determined the structural direction $h_m(\mathbf{x}; \boldsymbol{\theta}_m)$, a single-parameter line search determines the optimal stage step size γ_m :

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^N \mathcal{L} \left(y^{(i)}, f_{m-1}(\mathbf{x}^{(i)}) + \gamma h_m(\mathbf{x}^{(i)}; \boldsymbol{\theta}_m) \right) \quad (27)$$

When base components $h_m(\mathbf{x})$ are J -terminal node decision trees, the feature space is partitioned into J disjoint regions $\{R_{jm}\}_{j=1}^J$. Rather than enforcing a single global step size γ_m across the entire tree, Gradient Boosting optimizes an independent response value γ_{jm} for each terminal region R_{jm} :

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{\mathbf{x}^{(i)} \in R_{jm}} \mathcal{L} \left(y^{(i)}, f_{m-1}(\mathbf{x}^{(i)}) + \gamma \right) \quad (28)$$

The additive functional model updates as:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \sum_{j=1}^J \gamma_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \quad (29)$$

3.4 Derivation of Pseudo-Residuals for Common Loss Functions

1. Squared Error Loss (Regression: Mean Squared Error)

$$\mathcal{L}(y, f) = \frac{1}{2}(y - f)^2 \implies g_{im} = -\frac{\partial \mathcal{L}}{\partial f} = y^{(i)} - f_{m-1}(\mathbf{x}^{(i)}) \quad (30)$$

Pseudo-residuals reduce to standard residual errors.

2. Absolute Error Loss (Robust Regression: Mean Absolute Error)

$$\mathcal{L}(y, f) = |y - f| \implies g_{im} = -\frac{\partial \mathcal{L}}{\partial f} = \text{sign} \left(y^{(i)} - f_{m-1}(\mathbf{x}^{(i)}) \right) \quad (31)$$

Pseudo-residuals depend strictly on error sign, conferring robustness against extreme numerical outliers.

3. Binary Log-Loss (Classification: Bernoulli Negative Log-Likelihood)

Let $y \in \{0, 1\}$, and model probability $p = \sigma(f) = \frac{1}{1+e^{-f}}$:

$$\mathcal{L}(y, f) = -y \ln p - (1 - y) \ln(1 - p) = -yf + \ln(1 + e^f) \quad (32)$$

Taking derivative with respect to raw logit score f :

$$g_{im} = -\frac{\partial \mathcal{L}}{\partial f} = y^{(i)} - \frac{e^{f_{m-1}(\mathbf{x}^{(i)})}}{1 + e^{f_{m-1}(\mathbf{x}^{(i)})}} = y^{(i)} - p_{m-1}(\mathbf{x}^{(i)}) \quad (33)$$

Pseudo-residuals represent the residual difference between ground-truth binary targets and modeled probabilities.

Functional Gradient Descent Path in Function Space

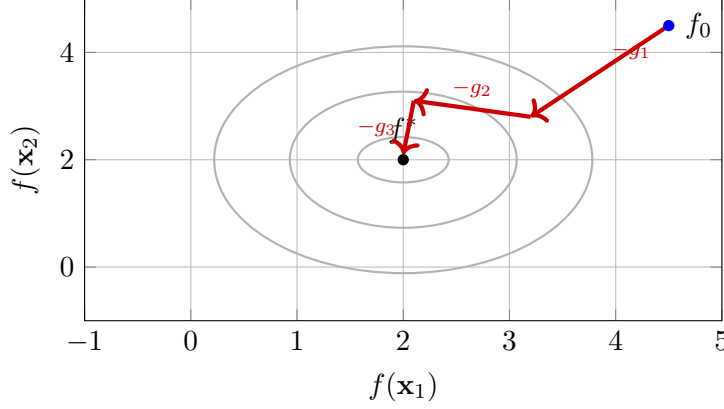


Figure 1: Trajectory of functional gradient descent. Iterative stage updates steer hypothesis f_m toward optimal function f^* along pseudo-residual vector paths.

4 Information-Theoretic View and Physical Analogies

4.1 Information-Theoretic View: Greedy KL-Divergence Minimization

Boosting can be understood through information theory as an iterative algorithm that minimizes the Kullback-Leibler (KL) divergence between empirical data targets and additive model predictions.

At each stage of AdaBoost, updating instance weights via $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y^{(i)} h_m(\mathbf{x}^{(i)}))$ alters the empirical probability distribution $\tilde{W}^{(m)}$.

Selecting $h_m(\mathbf{x})$ to minimize weighted error ϵ_m maximizes the **Mutual Information** $I(Y; h_m(\mathbf{X}))$ between base model predictions and true class labels under the current reweighted sample distribution. Boosting continuously warps the distribution metric to extract maximum non-redundant information from weak base hypotheses.

4.2 Physical Analogy: Rayleigh-Schrödinger Perturbation Theory

The structure of additive functional boosting mirrors **Rayleigh-Schrödinger Perturbation Theory** in quantum mechanics and theoretical physics.

Consider solving the time-independent Schrödinger wave equation for a complex physical system with Hamiltonian H :

$$H|\psi\rangle = E|\psi\rangle \quad (34)$$

When H cannot be solved analytically, physics decomposes H into an unperturbed solvable baseline operator H_0 plus a sequence of small perturbative fields λV :

$$H = H_0 + \lambda V_1 + \lambda^2 V_2 + \lambda^3 V_3 + \dots \quad (35)$$

The eigenstate wavefunction $|\psi\rangle$ and energy eigenvalues E are expanded as additive perturbative series:

$$|\psi\rangle = |\psi^{(0)}\rangle + \lambda|\psi^{(1)}\rangle + \lambda^2|\psi^{(2)}\rangle + \dots \quad (36)$$

$$E = E^{(0)} + \lambda E^{(1)} + \lambda^2 E^{(2)} + \dots \quad (37)$$

- The unperturbed baseline state $|\psi^{(0)}\rangle$ corresponds to the initial coarse prediction $f_0(\mathbf{x})$.
- Higher-order perturbative corrections $|\psi^{(k)}\rangle$ correspond to stage base learners $h_m(\mathbf{x})$.
- Each successive perturbation term does not attempt to solve the global physical system from scratch. Instead, it computes a localized corrective adjustment that cancels out residual energy field errors left behind by lower-order approximations.

5 Regularization, Shrinkage, and Generalization Dynamics

Because boosting minimizes loss stagewise, over-fitting can occur if M is excessively large or if base components overfit pseudo-residuals. Regularization techniques maintain generalization performance.

5.1 Shrinkage (Learning Rate Control)

Friedman introduced **Shrinkage** by adding a learning rate parameter $\nu \in (0, 1]$ to scale the contribution of each stage component:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu \cdot \sum_{j=1}^J \gamma_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \quad (38)$$

Decreasing learning rate ν ($\nu < 0.1$) acts as a numerical damper. Smaller values of ν require a larger number of total iterations M , but provide better empirical generalization error by forcing base learners to learn fine-grained corrective features.

5.2 Stochastic Gradient Boosting (Subsampling)

Inspired by bootstrap techniques, **Stochastic Gradient Boosting** (Friedman, 2002) samples a random fraction $\eta \in (0, 1)$ of the training data **without replacement** at each stage m .

The pseudo-residuals g_{im} and base learner $h_m(\mathbf{x})$ are constructed strictly using the subsampled dataset. This introduces random sampling noise into the functional gradient vector, helping the optimizer escape local minima and reducing inter-stage base model correlation.

5.3 Margin Maximization Theory

A theoretical puzzle in early boosting applications was that AdaBoost often continued to improve test generalization error long after training error had reached zero ($R_{\text{emp}} = 0$).

Schapire et al. (1998) explained this using **Margin Theory**. Define the normalized classification margin $yf(\mathbf{x})/\|\boldsymbol{\alpha}\|_1$:

Definition 5.1 (Classification Margin). For binary classification where $f(\mathbf{x}) = \sum \alpha_m h_m(\mathbf{x})$, the normalized margin for sample $(\mathbf{x}^{(i)}, y^{(i)})$ is:

$$\text{Margin}(\mathbf{x}^{(i)}, y^{(i)}) = \frac{y^{(i)} \sum_{m=1}^M \alpha_m h_m(\mathbf{x}^{(i)})}{\sum_{m=1}^M \alpha_m} \in [-1, +1] \quad (39)$$

Even after classification error drops to zero (Margin > 0), additional boosting iterations increase the magnitude of small positive margins, pushing training points further away from the decision boundary.

Theorem 5.1 (Schapire Margin Bound). With probability $1 - \delta$, the generalization error is upper-bounded by the fraction of training margins falling below threshold $\theta > 0$:

$$P_{\text{test}}(yf(\mathbf{x}) \leq 0) \leq P_{\text{train}}(yf(\mathbf{x}) \leq \theta) + \mathcal{O}\left(\sqrt{\frac{d}{\theta^2 N} \log^2\left(\frac{N}{d}\right) + \log\left(\frac{1}{\delta}\right)}\right) \quad (40)$$

where d is the VC-dimension of base hypothesis class $\mathcal{H}$.

Continued boosting increases parameter θ , tightening the statistical generalization bound.

6 Production-Grade Vectorized NumPy Implementation

The following Python code implements a production-grade Boosting framework from scratch, incorporating AdaBoost Classification and Gradient Boosting Regression/Classification with functional line search, shrinkage, and stochastic subsampling.

```
1 import numpy as np
2
3 class DecisionStump:
4     """Weak base learner: 1-depth Decision Stump for classification/
5     regression."""
6     def __init__(self):
7         self.feature_idx = None
8         self.threshold = None
9         self.polarity = 1
10        self.alpha = None
11
12    def fit_classification(self, X: np.ndarray, y: np.ndarray,
13    sample_weights: np.ndarray):
14        N, d = X.shape
15        min_error = float('inf')
16
17        for feature in range(d):
18            X_column = X[:, feature]
19            thresholds = np.unique(X_column)
20
21            for threshold in thresholds:
22                for polarity in [1, -1]:
23                    predictions = np.ones(N)
24                    if polarity == 1:
25                        predictions[X_column < threshold] = -1
26                    else:
27                        predictions[X_column > threshold] = -1
28
29                    # Weighted error
30                    error = np.sum(sample_weights[y != predictions])
31
32                    if error < min_error:
33                        min_error = error
34                        self.polarity = polarity
35                        self.threshold = threshold
36                        self.feature_idx = feature
37
38        return self
39
40    def predict_classification(self, X: np.ndarray) -> np.ndarray:
41        N = X.shape[0]
42        X_column = X[:, self.feature_idx]
43        predictions = np.ones(N)
44        if self.polarity == 1:
45            predictions[X_column < self.threshold] = -1
46        else:
47            predictions[X_column > self.threshold] = -1
48        return predictions
49
50 class AdaBoostClassifierEngine:
51     """
52     Production-grade AdaBoost binary classifier minimizing exponential loss
53     via sequential sample weight updates.
```

```

53     """
54     def __init__(self, n_estimators: int = 50):
55         self.n_estimators = n_estimators
56         self.estimators = []
57         self.estimator_weights = []
58
59     def fit(self, X: np.ndarray, y: np.ndarray):
60         """Fits AdaBoost model over binary targets y in {-1, +1}."""
61         N, d = X.shape
62         # Initialize uniform sample weights
63         w = np.full(N, 1.0 / N)
64
65         for _ in range(self.n_estimators):
66             stump = DecisionStump()
67             stump.fit_classification(X, y, w)
68             predictions = stump.predict_classification(X)
69
70             # Weighted error rate epsilon_m
71             misclassified = (y != predictions)
72             error = np.sum(w[misclassified]) / np.sum(w)
73
74             # Avoid division by zero
75             error = np.clip(error, 1e-10, 1.0 - 1e-10)
76
77             # Stage weight alpha_m = 0.5 * ln((1 - error) / error)
78             alpha = 0.5 * np.log((1.0 - error) / error)
79
80             # Weight update: w_i = w_i * exp(-alpha * y_i * h(x_i))
81             w *= np.exp(-alpha * y * predictions)
82             w /= np.sum(w) # Normalize
83
84             self.estimators.append(stump)
85             self.estimator_weights.append(alpha)
86
87         return self
88
89     def predict(self, X: np.ndarray) -> np.ndarray:
90         """Predicts binary class labels in {-1, +1} via stage-weighted
91         votes."""
92         stump_preds = np.array([stump.predict_classification(X) for stump
93         in self.estimators])
94         weights = np.array(self.estimator_weights)[: , np.newaxis]
95         raw_scores = np.sum(weights * stump_preds, axis=0)
96         return np.sign(raw_scores)
97
98 class GradientBoostingEngine:
99     """
100     Production-grade Gradient Boosting Engine for Regression minimizing MSE
101     loss
102     via functional gradient descent on pseudo-residuals.
103     """
104     def __init__(self, n_estimators: int = 100, learning_rate: float = 0.1,
105                   max_depth: int = 3, subsample: float = 1.0):
106         self.n_estimators = n_estimators
107         self.lr = learning_rate
108         self.max_depth = max_depth
109         self.subsample = subsample
110
111         self.f0 = 0.0

```

```

110     self.trees = []
111
112     def fit(self, X: np.ndarray, y: np.ndarray):
113         """Fits additive model to Continuous Targets using MSE loss."""
114         N, d = X.shape
115         self.f0 = np.mean(y) # Optimal initial constant hypothesis
116         f_current = np.full(N, self.f0)
117
118         for m in range(self.n_estimators):
119             # 1. Compute Pseudo-Residuals  $g_i = -(dL/df) = y_i - f_{m-1}(x_i)$ 
120             pseudo_residuals = y - f_current
121
122             # 2. Stochastic Subsampling (if enabled)
123             if self.subsample < 1.0:
124                 indices = np.random.choice(N, int(self.subsample * N),
125 replace=False)
126                 X_train, res_train = X[indices], pseudo_residuals[indices]
127             else:
128                 X_train, res_train = X, pseudo_residuals
129
130             # 3. Fit Decision Stump/Tree to Pseudo-Residuals
131             stump = DecisionStump()
132             stump.fit_classification(X_train, np.sign(res_train), np.ones(
133 len(res_train)))
134
135             # Predict residual direction
136             h_pred = stump.predict_classification(X)
137
138             # 4. Line Search for optimal step gamma
139             gamma = np.sum(pseudo_residuals * h_pred) / (np.sum(h_pred**2)
140 + 1e-10)
141             stump.alpha = gamma
142
143             # 5. Update functional model:  $f_m = f_{m-1} + lr * gamma * h_m(x)$ 
144             f_current += self.lr * gamma * h_pred
145             self.trees.append(stump)
146
147         return self
148
149     def predict(self, X: np.ndarray) -> np.ndarray:
150         """Evaluates continuous predictions via functional additive updates
151 ."""
152         N = X.shape[0]
153         f_pred = np.full(N, self.f0)
154
155         for stump in self.trees:
156             h_pred = stump.predict_classification(X)
157             f_pred += self.lr * stump.alpha * h_pred
158
159         return f_pred
160
161 if __name__ == "__main__":
162     np.random.seed(42)
163
164     # 1. Validation Run: AdaBoost Binary Classifier
165     N_samples, d_features = 300, 5
166     X_cls = np.random.randn(N_samples, d_features)
167     y_cls = np.where(X_cls[:, 0] + X_cls[:, 1]*0.8 > 0, 1, -1)

```

```

166     adaboost = AdaBoostClassifierEngine(n_estimators=30)
167     adaboost.fit(X_cls, y_cls)
168     preds_cls = adaboost.predict(X_cls)
169
170     acc = np.mean(preds_cls == y_cls)
171     print(f"=== ADABOOST CLASSIFICATION ACCURACY: {acc * 100:.2f}% ===")
172
173     # 2. Validation Run: Gradient Boosting Regressor
174     y_reg = X_cls[:, 0]**2 + np.sin(X_cls[:, 1]) + 0.05 * np.random.randn(
N_samples)
175     gbdt = GradientBoostingEngine(n_estimators=50, learning_rate=0.08,
subsample=0.8)
176     gbdt.fit(X_cls, y_reg)
177     preds_reg = gbdt.predict(X_cls)
178
179     mse = np.mean((preds_reg - y_reg)**2)
180     print(f"=== GRADIENT BOOSTING REGRESSION MSE: {mse:.4f} ===")

```

Listing 1: Vectorized NumPy implementation of AdaBoost and Gradient Boosting Engine.